ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & THUẬT MÁY TÍNH

XỬ LÝ TÍN HIỆU SỐ (CO2036)

LAB 1

Digital Signal Processing with Scilab: Sampling and Quantization Analysis

Lớp: L03 - Học Kỳ HK252
Ngày nộp: 12/03/2026

GVHD: Thầy Phạm Công Thái

Sinh Viên	MSSV
Phạm Công Võ	2313946
Nguyễn Anh Thư	2313385

TP.Hồ Chí Minh, Tháng 03/2026

PHÂN CÔNG NHIỆM VỤ VÀ KẾT QUẢ THỰC HIỆN ĐỀ TÀI CỦA TỪNG THÀNH VIÊN NHÓM

STT	Họ và tên	MSSV	Nhiệm vụ	Kết quả	Chữ ký
1	Nguyễn Anh Thư	2313385	• Hiện thực bài tập 1 và 2 • Viết báo cáo	100%	
2	Phạm Công Võ	2313946	• Hiện thực bài tập 1 và 2 • Viết báo cáo	100%	

Mục lục

1 Giới thiệu **4**

2 Mục tiêu **4**

3 Phương pháp thực hiện **4**

4 Bài Tập Thực Hành **5**

4.1 Exercise 1.1: Phép Toán Trên Vector và Ma Trận 5

4.2 Exercise 1.2: Phân tích tín hiệu 8

Tài liệu tham khảo **18**

Danh sách hình vẽ

1 Đồ thị vector x+1 5
2 Đồ thị vector x.y 6
3 Đồ thị $\sin(x)$ trên đoạn $[0, \pi]$ 7
4 Đồ thị tín hiệu tương tự $x_a(t)$ trong 5 chu kỳ 9
5 Đồ thị tín hiệu rời rạc $x(n)$ trong 5 chu kỳ 13
6 Đồ thị tín hiệu lượng tử hóa $x_q(n)$ trong 5 chu kỳ 15
7 Biểu diễn các tín hiệu: $x_a(t)$, $x(n)$ và $x_q(n)$ trong cùng một cửa sổ đồ thị 17

1 Giới thiệu

Trong bối cảnh công nghệ số phát triển mạnh mẽ, **Xử lý tín hiệu số (Digital Signal Processing)** đóng vai trò quan trọng trong nhiều lĩnh vực kỹ thuật hiện đại như viễn thông, thiết bị y sinh và các hệ thống điều khiển thông minh. Các hệ thống này đều dựa trên khả năng thu thập, phân tích và xử lý tín hiệu một cách chính xác và hiệu quả.

Để hỗ trợ việc học tập và thực hành trong lĩnh vực này, sinh viên cần sử dụng các công cụ tính toán khoa học phục vụ cho việc mô phỏng và phân tích tín hiệu. **Scilab** là một phần mềm mã nguồn mở mạnh mẽ, được thiết kế cho tính toán số, xử lý ma trận và trực quan hóa dữ liệu, với cú pháp tương đồng MATLAB.

Bài thực hành số 1 nhằm giúp sinh viên làm quen với môi trường Scilab và các thao tác cơ bản trong xử lý tín hiệu với các mục tiêu chính sau:

- Cài đặt và làm quen với phần mềm Scilab
- Nghiên cứu tài liệu *Scilab for Beginners*
- Thực hành các phép toán trên vector và ma trận
- Mô phỏng và trực quan hóa tín hiệu trong quá trình chuyển đổi Tương tự - Số (ADC)

Thông qua bài thực hành này, sinh viên bước đầu hình thành tư duy lập trình khoa học và kỹ năng mô phỏng tín hiệu - nền tảng quan trọng cho các môn học chuyên ngành tiếp theo.

2 Mục tiêu

Bài thực hành này nhằm giúp sinh viên làm quen với môi trường lập trình Scilab và các thao tác cơ bản trong xử lý tín hiệu số. Thông qua các bài tập, sinh viên có thể hiểu rõ cách biểu diễn và phân tích tín hiệu trong miền thời gian, cũng như các bước cơ bản trong quá trình chuyển đổi từ tín hiệu tương tự sang tín hiệu số.

Các mục tiêu chính của bài thực hành bao gồm:

- Làm quen với phần mềm Scilab và môi trường lập trình tính toán khoa học.
- Thực hiện các phép toán trên vector và ma trận trong Scilab.
- Mô phỏng và trực quan hóa tín hiệu bằng các công cụ vẽ đồ thị.
- Phân tích quá trình lấy mẫu và lượng tử hóa tín hiệu trong hệ thống chuyển đổi ADC.

3 Phương pháp thực hiện

- Nghiên cứu tài liệu hướng dẫn *Scilab for Beginners* để nắm các cú pháp cơ bản của Scilab.
- Sử dụng Scilab để thực hiện các phép toán trên vector và ma trận.
- Xây dựng chương trình mô phỏng tín hiệu tương tự, tín hiệu rời rạc và tín hiệu lượng tử hóa.
- Sử dụng các hàm vẽ đồ thị trong Scilab để trực quan hóa và phân tích các tín hiệu thu được.

4 Bài Tập Thực Hành

Sử dụng phần mềm Scilab để thực hiện các bài tập sau.

4.1 Exercise 1.1: Phép Toán Trên Vector và Ma Trận

Yêu cầu đề bài

- **Tạo một vector có dạng** $(x_1+1, x_2+1, x_3+1, x_4+1)$**, trong đó** x_1, x_2, x_3, x_4 **là các phần tử của vector** $x = 1:4$**.**
- **Tạo một vector có dạng** $(x_1y_1, x_2y_2, x_3y_3, x_4y_4)$**, trong đó** $x_1, \ldots, x_4$ **và** $y_1, \ldots, y_4$ **lần lượt là các phần tử của hai vector** $x = 1:4$ **và** $y = 5:8$**.**
- **Tạo một vector có dạng** $(\sin(x_1), \sin(x_2), \ldots, \sin(x_{10}))$**, trong đó** x **là một vector gồm 10 giá trị được chọn tuyến tính trong khoảng** $[0, \pi]$**.**

Bài 1.1.a - Tạo vector dịch chuyển $(x+1)$

Phân tích:

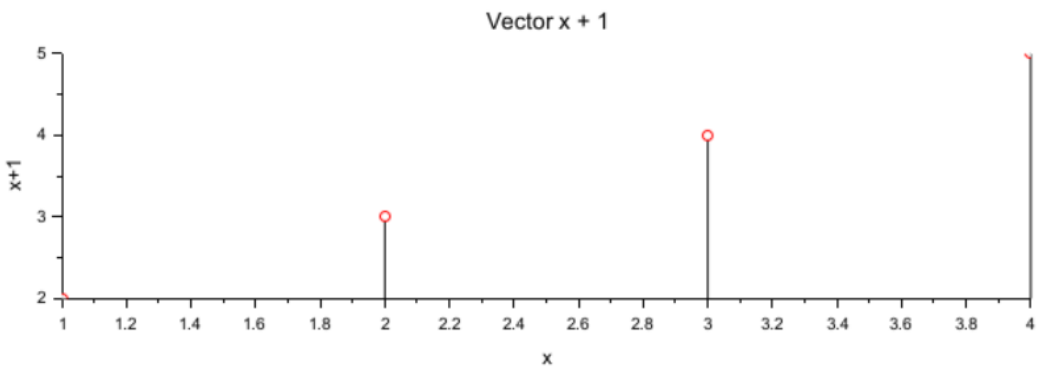
Bài toán thực hiện phép *cộng vô hướng* giữa một vector và một hằng số. Trong Scilab, phép toán này được áp dụng trực tiếp cho từng phần tử của vector mà không cần sử dụng vòng lặp.

Mã lệnh Scilab:

```
x1 = 1 : 4;
result1 = x1 + 1;

disp(result1)
subplot(3, 1, 1);
plot2d3(x1, result1);
plot(x1, result1, "ro");
title("Vector x + 1");
xlabel("x"); ylabel("x+1");
```

Kết quả:

```
result = 2   3   4   5
```



Hình 1: *Đồ thị vector x+1*

Bài 1.1.b - Nhân từng phần tử hai vector

Phân tích:

Bài toán yêu cầu thực hiện phép nhân tương ứng giữa các phần tử cùng vị trí của hai vector x và y. Đây là phép nhân *element-wise multiplication* (còn gọi là phép nhân Hadamard). Trong Scilab, phép toán này được thực hiện bằng toán tử `.*`, khác với toán tử `*` dùng cho phép nhân ma trận.

Mã lệnh Scilab:

```
x2 = 1 : 4;
y2 = 5 : 8;
result2 = x2 .* y2;

disp(result2)
subplot(3, 1, 2);
plot2d3(x2, result2);
plot(x2, result2, "gs");
title("Vector x .* y");
xlabel("x"); ylabel("x*y");
```

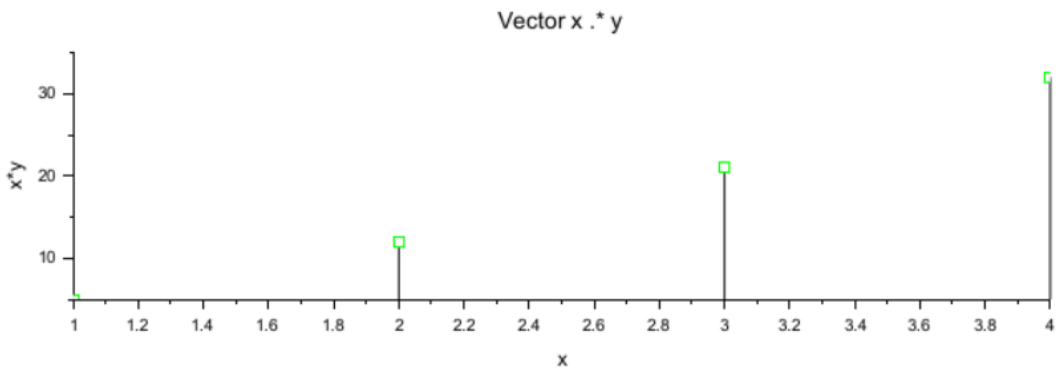
Kết quả:

```
result = 5   12   21   32
```

Trong phép nhân theo từng phần tử, mỗi phần tử của vector x được nhân với phần tử tương ứng của vector y theo cùng vị trí. Cụ thể:

Phần tử	**x**	**y**	$x \times y$
1	1	5	5
2	2	6	12
3	3	7	21
4	4	8	32

Lưu ý rằng toán tử `.*` được sử dụng để thực hiện phép nhân theo từng phần tử. Nếu sử dụng toán tử `*` thay cho `.*`, Scilab sẽ hiểu đó là phép nhân ma trận và có thể gây lỗi do hai vector hàng không phù hợp để thực hiện phép nhân ma trận trực tiếp.



Hình 2: *Đồ thị vector x.y*

Bài 1.1.c - Áp dụng hàm sin() trên vector tuyến tính

Phân tích:

Bài toán yêu cầu tạo một vector gồm các giá trị $\sin(x_i)$, trong đó x là vector gồm 10 điểm được phân bố đều trên đoạn $[0, \pi]$. Quá trình thực hiện gồm hai bước: (1) sinh vector tuyến tính bằng hàm `linspace()`, và (2) áp dụng hàm lượng giác `sin()` lên toàn bộ vector theo cơ chế tính toán theo từng phần tử.

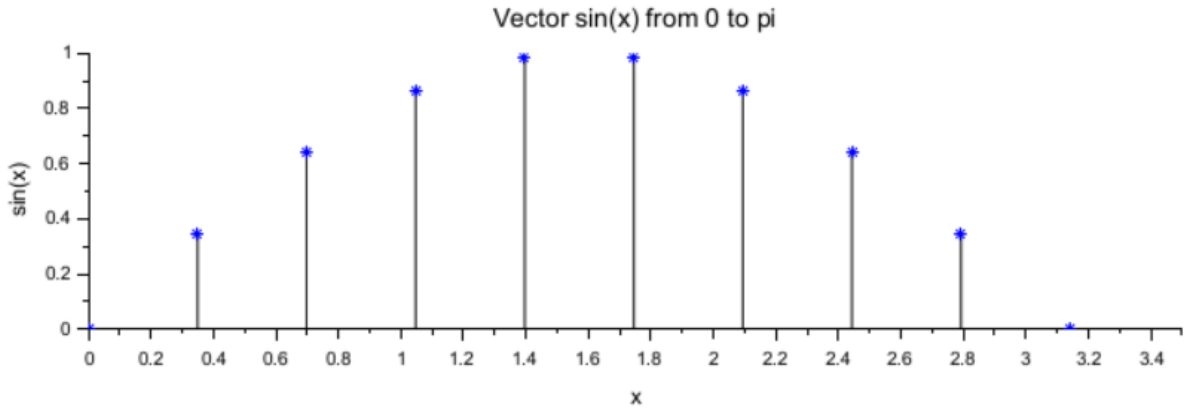
Mã lệnh Scilab:

```
x3 = linspace(0, %pi, 10);
result3 = sin(x3);

disp(result3)
subplot(3, 1, 3);
plot2d3(x3, result3);
plot(x3, result3, "b*");
title("Vector sin(x) from 0 to pi");
xlabel("x"); ylabel("sin(x)");
```

Kết quả:

```
x      = 0   0.349   0.698   1.047   1.396   1.745   2.094   2.443   2.793
    3.142
result = 0.000   0.342   0.643   0.866   0.985   0.985   0.866   0.643   0.342
    0.000
```

Hàm `linspace(0, %pi, 10)` tạo ra 10 giá trị cách đều nhau trên đoạn $[0, \pi]$, bao gồm cả hai điểm đầu và cuối của khoảng. Sau đó, hàm `sin()` được áp dụng lên toàn bộ vector x để tạo vector kết quả *result*. Kết quả cho thấy đặc tính đối xứng của hàm $\sin(x)$ trên đoạn $[0, \pi]$: giá trị tăng dần từ 0 đến cực đại tại $x = \pi/2$, sau đó giảm đối xứng về 0 tại $x = \pi$. Đồ thị được vẽ bằng lệnh `plot()` giúp trực quan hóa các điểm dữ liệu được lấy mẫu trên nửa chu kỳ của sóng sine.



Hình 3: *Đồ thị* $\sin(x)$ *trên đoạn* $[0, \pi]$

4.2 Exercise 1.2: Phân tích tín hiệu

Yêu cầu đề bài

Xét tín hiệu tương tự sau:

$$x_a(t) = 3\sin(100\pi t)$$

- **Sử dụng Scilab để vẽ tín hiệu $x_a(t)$ trong 5 chu kỳ.**
- **Xác định tín hiệu rời rạc $x(n)$ khi tín hiệu $x_a(t)$ được lấy mẫu với tần số lấy mẫu $F_s = 300$ mẫu/giây.**
- **Khảo sát tính tuần hoàn của tín hiệu $x(n)$. Nếu $x(n)$ là tín hiệu tuần hoàn, hãy xác định tần số và chu kỳ của $x(n)$. Sau đó, sử dụng Scilab để vẽ $x(n)$ trong 5 chu kỳ.**
- **Xác định tín hiệu lượng tử hóa $x_q(n)$ với bước lượng tử $\Delta = 0.1$ bằng phương pháp cắt ngắn (truncated method). Sau đó, vẽ tín hiệu $x_q(n)$ trong 5 chu kỳ.**

  Lưu ý: Tất cả các đồ thị bao gồm $x_a(t)$, $x(n)$ và $x_q(n)$ phải được hiển thị trong cùng một cửa sổ.

Phần 1 - Phân tích tín hiệu tương tự $x_a(t)$

<u>Phân tích:</u>
Xét tín hiệu tương tự:

$$x_a(t) = 3\sin(100\pi t)$$

So sánh với dạng tổng quát của tín hiệu hình sin:

$$x_a(t) = A\sin(2\pi f_0 t + \varphi)$$

ta xác định được các thông số đặc trưng của tín hiệu như sau:

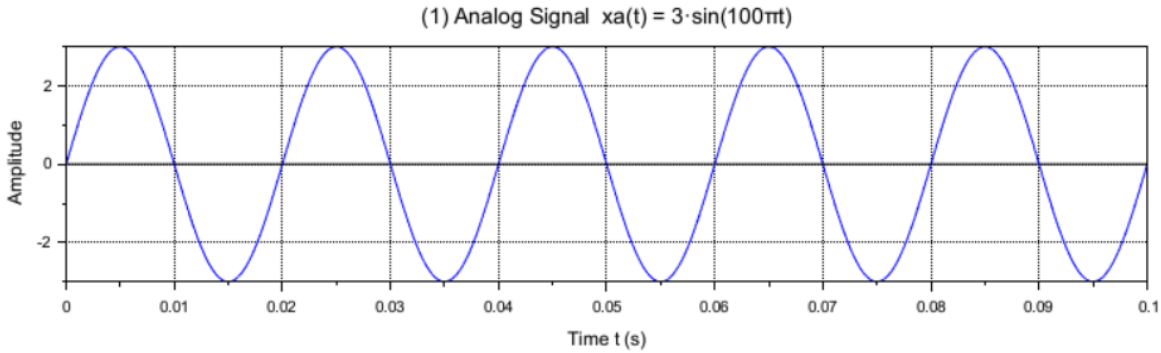
Thông số	Ký hiệu	Giá trị	Đơn vị
Biên độ	A	3	–
Tần số góc	Ω_0	100π	rad/s
Tần số	$f_0 = \Omega_0/2\pi$	50	Hz
Chu kỳ	$T_0 = 1/f_0$	0.02	s
Pha ban đầu	φ	0	rad

Để quan sát tín hiệu trong **5 chu kỳ**, ta xét khoảng thời gian:

$$t \in [0, 5T_0] = [0, 0.1] \text{ s}$$

Mã lệnh Scilab:

```
F = 50;            // Signal frequency (Hz)
Fs = 300;          // Sampling frequency (Hz)
T = 1 / F;         // Signal period (s)
Ts = 1 / Fs;       // Sampling period (s)
time = 5 * T;      // Total time: 5 periods
delta = 0.1;       // Quantization step

// 1. Analog Signal xa(t)
t = linspace(0, time, 1000);
xa = 3 * sin(2 * %pi * F * t);

subplot(3, 1, 1);
plot(t, xa, 'b');
plot(t, zeros(t), 'k-');      // Zero reference line (y = 0)
title('(1) Analog Signal xa(t)');
xlabel('Time t (s)');
ylabel('Amplitude');
xgrid();
```



Hình 4: *Đồ thị tín hiệu tương tự $x_a(t)$ trong 5 chu kỳ*

Phần 2 - Xác định tín hiệu rời rạc $x(n)$

<u>Phân tích:</u>

Giả sử tín hiệu $x_a(t)$ được lấy mẫu với tần số lấy mẫu:

$$F_s = 300 \text{ mẫu/s}$$

Kiểm tra điều kiện Nyquist:

Để tránh hiện tượng chồng phổ (aliasing), tần số lấy mẫu phải thỏa mãn:

$$\boldsymbol{F_s \geq 2f_0}$$

$$\Rightarrow 300 \geq 2 \times 50 = 100$$

Điều kiện Nyquist thỏa mãn, do đó tín hiệu Analog có thể được khôi phục chính xác từ các mẫu rời rạc.

Chu kỳ lấy mẫu được xác định bởi:

$$T_s = \frac{1}{F_s} = \frac{1}{300} \text{ s}$$

Tín hiệu rời rạc thu được bằng cách thay $t = nT_s$ vào $x_a(t)$:

$$\boldsymbol{x(n) = x_a(nT_s)}$$

$$\Rightarrow x(n) = 3\sin(100\pi n T_s)$$

$$\Rightarrow x(n) = 3\sin\left(\frac{100\pi n}{300}\right)$$

Rút gọn:

$$\Rightarrow x(n) = 3\sin\left(\frac{\pi n}{3}\right)$$

Do đó tín hiệu rời rạc có dạng:

$$\boldsymbol{x(n) = 3\sin\left(\frac{\pi n}{3}\right)}$$

Tần số góc rời rạc của tín hiệu là:

$$\boldsymbol{\omega_0 = 2\pi\frac{f_0}{F_s}}$$

$$\Rightarrow \omega_0 = 2\pi\frac{50}{300} = \frac{\pi}{3} \text{ rad/mẫu}$$

Phần 3 - Xác định tính tuần hoàn và vẽ tín hiệu rời rạc $x(n)$

<u>Phân tích:</u>

Theo lý thuyết tín hiệu rời rạc, tín hiệu dạng

$$x(n) = A\sin(2\pi f n)$$

là **tuần hoàn** khi và chỉ khi tần số chuẩn hóa f là một số hữu tỉ, tức là:

$$\boldsymbol{f = \frac{k}{N}, \qquad k \in \mathbb{Z},\ N \in \mathbb{Z}^+}$$

Khi điều kiện trên được thỏa mãn, tín hiệu sẽ lặp lại sau một số mẫu hữu hạn. Chu kỳ cơ bản N_0 được xác định bởi:

$$\boldsymbol{N_0 = \frac{k'}{f}, \quad \min|k'| \in \mathbb{Z} \text{ sao cho } \frac{k'}{f} \in \mathbb{Z}^+}$$

Note: Khi tín hiệu rời rạc được tạo thành từ tổng của nhiều tín hiệu tuần hoàn có chu kỳ cơ bản lần lượt là $N_1, N_2, \ldots, N_k$, thì chu kỳ cơ bản của tín hiệu tổng sẽ bằng bội số chung nhỏ nhất của các chu kỳ thành phần:

$$\boldsymbol{N_0 = \mathrm{LCM}(N_1, N_2, \ldots, N_k)}$$

Xác định tần số chuẩn hóa:

Từ kết quả ở Phần 2, tín hiệu rời rạc thu được là:

$$x(n) = 3\sin\left(\frac{\pi n}{3}\right)$$

So sánh với dạng chuẩn:

$$x(n) = A\sin(2\pi f n)$$

ta có:

$$\omega = 2\pi f = \frac{\pi}{3}$$

$$\Rightarrow \boldsymbol{f = \frac{\omega}{2\pi} = \frac{1}{6}}$$

Kiểm tra tính tuần hoàn:

$$f = \frac{1}{6} = \frac{k}{N}, \quad \text{với } k = 1,\ N = 6 \in \mathbb{Z}^+$$

Do f là một số hữu tỉ nên tín hiệu $x(n)$ là **tín hiệu tuần hoàn**.

Xác định chu kỳ cơ bản:
Áp dụng công thức:

$$N_0 = \frac{k'}{f}$$

Thay $f = \frac{1}{6}$ ta được:

$$N_0 = \frac{k'}{1/6} = 6k'$$

Chọn $k' = 1$ là giá trị nguyên dương nhỏ nhất để N_0 thuộc tập số nguyên dương. Khi đó ta thu được:

$$\boldsymbol{N_0 = 6} \text{ mẫu}$$

Như vậy tín hiệu rời rạc sẽ lặp lại sau mỗi 6 mẫu.

Xác minh kết quả:
Xét:

$$x(n+6) = 3\sin\left(\frac{\pi(n+6)}{3}\right) = 3\sin\left(\frac{\pi n}{3} + 2\pi\right) = 3\sin\left(\frac{\pi n}{3}\right) = x(n)$$

Nhận xét:

Biên độ cực đại của tín hiệu rời rạc chỉ đạt khoảng ± 2.598, nhỏ hơn giá trị biên độ cực đại của tín hiệu liên tục là ± 3. Nguyên nhân là do các điểm lấy mẫu không trùng với các vị trí mà hàm sin đạt giá trị cực đại (tại các góc $90°$ và $270°$). Vì vậy, các mẫu thu được không đạt đúng đỉnh của tín hiệu liên tục.

Bảng thông số của tín hiệu rời rạc

Bảng 3.6. Thông số tín hiệu rời rạc

Thông số	Ký hiệu	Giá trị	Đơn vị
Tần số góc rời rạc	ω	$\frac{\pi}{3} \approx 1.047$	rad/mẫu
Tần số chuẩn hóa	f	$\frac{1}{6}$	chu kỳ/mẫu
Giá trị k tối giản	k'	1	–
Chu kỳ cơ bản	N_0	6	mẫu
Số mẫu của 5 chu kỳ	$5N_0$	30	mẫu

Bảng giá trị tín hiệu trong một chu kỳ

Để quan sát đặc điểm của tín hiệu rời rạc, ta tính giá trị của $x(n)$ trong một chu kỳ cơ bản $N_0 = 6$.

Bảng 3.7. Giá trị tín hiệu trong một chu kỳ

n	**Góc** $\frac{\pi n}{3}$	$\sin(\frac{\pi n}{3})$	$x(n) = 3\sin(\frac{\pi n}{3})$
0	0° (0)	0	0.000
1	60° ($\frac{\pi}{3}$)	$\frac{\sqrt{3}}{2} \approx 0.866$	2.598
2	120° ($\frac{2\pi}{3}$)	$\frac{\sqrt{3}}{2} \approx 0.866$	2.598
3	180° (π)	0	0.000
4	240° ($\frac{4\pi}{3}$)	$-\frac{\sqrt{3}}{2} \approx -0.866$	-2.598
5	300° ($\frac{5\pi}{3}$)	$-\frac{\sqrt{3}}{2} \approx -0.866$	-2.598

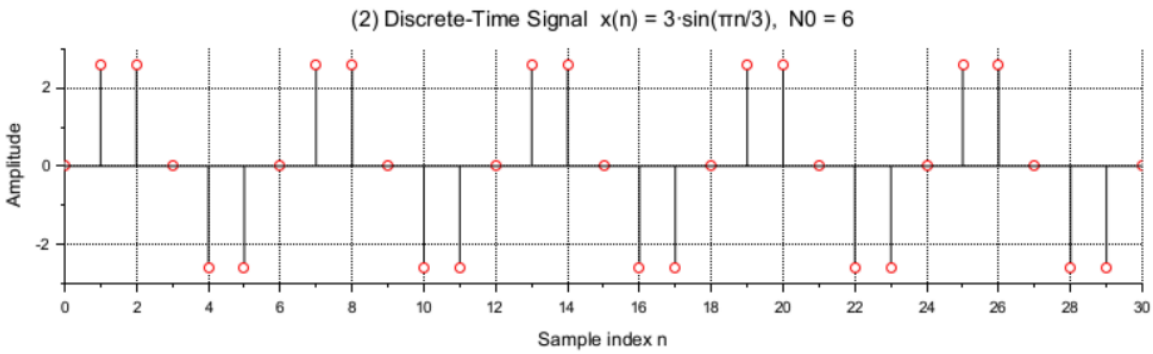
Nhận xét:

Từ bảng giá trị có thể thấy rằng tín hiệu rời rạc lặp lại sau mỗi 6 mẫu, phù hợp với chu kỳ cơ bản đã xác định $N_0 = 6$.

Ngoài ra, biên độ cực đại của tín hiệu chỉ đạt khoảng ± 2.598, nhỏ hơn biên độ cực đại ± 3 của tín hiệu liên tục. Nguyên nhân là do các thời điểm lấy mẫu không trùng với các vị trí mà hàm sin đạt giá trị cực đại (tại các góc 90° và 270°). Đây là hiện tượng thường gặp khi số mẫu trong một chu kỳ là hữu hạn.

Mã lệnh Scilab:

```
F = 50;            // Signal frequency (Hz)
Fs = 300;          // Sampling frequency (Hz)
T = 1 / F;         // Signal period (s)
Ts = 1 / Fs;       // Sampling period (s)
time = 5 * T;      // Total time: 5 periods
delta = 0.1;       // Quantization step

//  2. Discrete-Time Signal x(n)
// Sampling: x(n) = xa(nTs)
// Periodicity: f = F/Fs = 1/6  -  N0 = 6 samples/cycle
n = 0:30;
xn = 3 * sin(2 * %pi * F * n * Ts);

subplot(3, 1, 2);
plot2d3(n, xn);                          // Stem plot
plot(n, xn, 'ro');                       // Red dots at each sample
plot(n, zeros(n), 'k-');               // Zero reference line (y = 0)
title('(2)Discrete-Time Signal x(n), N0 = 6');
xlabel('Sample index n');
ylabel('Amplitude');
xgrid();
```



Hình 5: *Đồ thị tín hiệu rời rạc $x(n)$ trong 5 chu kỳ*

Phần 4 - Tín Hiệu Lượng Tử Hóa $x_q(n)$ với $\Delta = 0.1$ (Truncation)

Phương pháp Truncation (Cắt bỏ)

Trong quá trình lượng tử hóa, tín hiệu rời rạc có biên độ liên tục được chuyển thành tín hiệu có biên độ rời rạc bằng cách ánh xạ mỗi mẫu về một mức lượng tử xác định.

Với phương pháp **Truncation**, giá trị lượng tử được xác định bằng cách cắt bỏ phần thập phân theo hướng về âm vô cực. Khi đó tín hiệu lượng tử hóa được xác định bởi:

$$\boldsymbol{x_q(n) = \Delta \left\lfloor \frac{x(n)}{\Delta} \right\rfloor}$$

Trong đó:

- $\Delta = 0.1$ là bước lượng tử (quantization step)
- $\lfloor \cdot \rfloor$ là hàm *floor*, lấy phần nguyên theo hướng âm vô cực

Sai số lượng tử hóa

Sai số lượng tử hóa được xác định bởi:

$$\boldsymbol{e_q(n) = x_q(n) - x(n)}$$

Đối với phương pháp Truncation, sai số luôn thỏa mãn:

$$\boldsymbol{|e_q(n)| < \Delta}$$

Điều này có nghĩa là giá trị lượng tử hóa $x_q(n)$ luôn nhỏ hơn hoặc bằng giá trị tín hiệu ban đầu $x(n)$.

Bảng giá trị tín hiệu lượng tử hóa

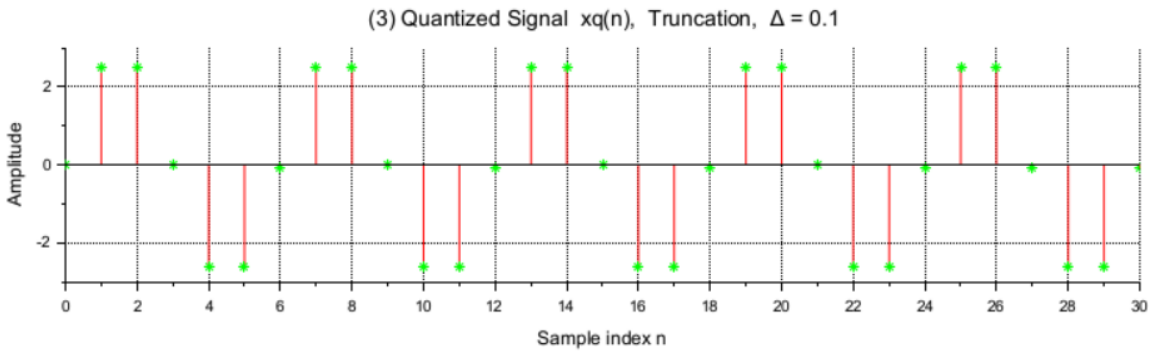
Các giá trị của tín hiệu lượng tử hóa $x_q(n)$ với $\Delta = 0.1$ trong một chu kỳ được trình bày trong Bảng dưới đây.

Bảng 4.1. Giá trị tín hiệu lượng tử hóa

n	$x(n)$	$x_q(n)$
0	0.000	0.0
1	2.598	2.5
2	2.598	2.5
3	0.000	0.0
4	-2.598	-2.6
5	-2.598	-2.6

Mã lệnh Scilab:

```
F = 50;            // Signal frequency (Hz)
Fs = 300;          // Sampling frequency (Hz)
T = 1 / F;         // Signal period (s)
Ts = 1 / Fs;       // Sampling period (s)
time = 5 * T;      // Total time: 5 periods
delta = 0.1;       // Quantization step

// 3. Quantized Signal xq(n)
xq = floor(xn / delta) * delta;

subplot(3, 1, 3);
plot2d3(n, xq, style=5);                // Stem plot
plot(n, xq, 'g*');                      // Green stars at each sample
plot(n, zeros(n), 'k-');               // Zero reference line (y = 0)
title('(3) Quantized Signal xq(n), Truncation, delta = 0.1');
xlabel('Sample index n');
ylabel('Amplitude');
xgrid();
```



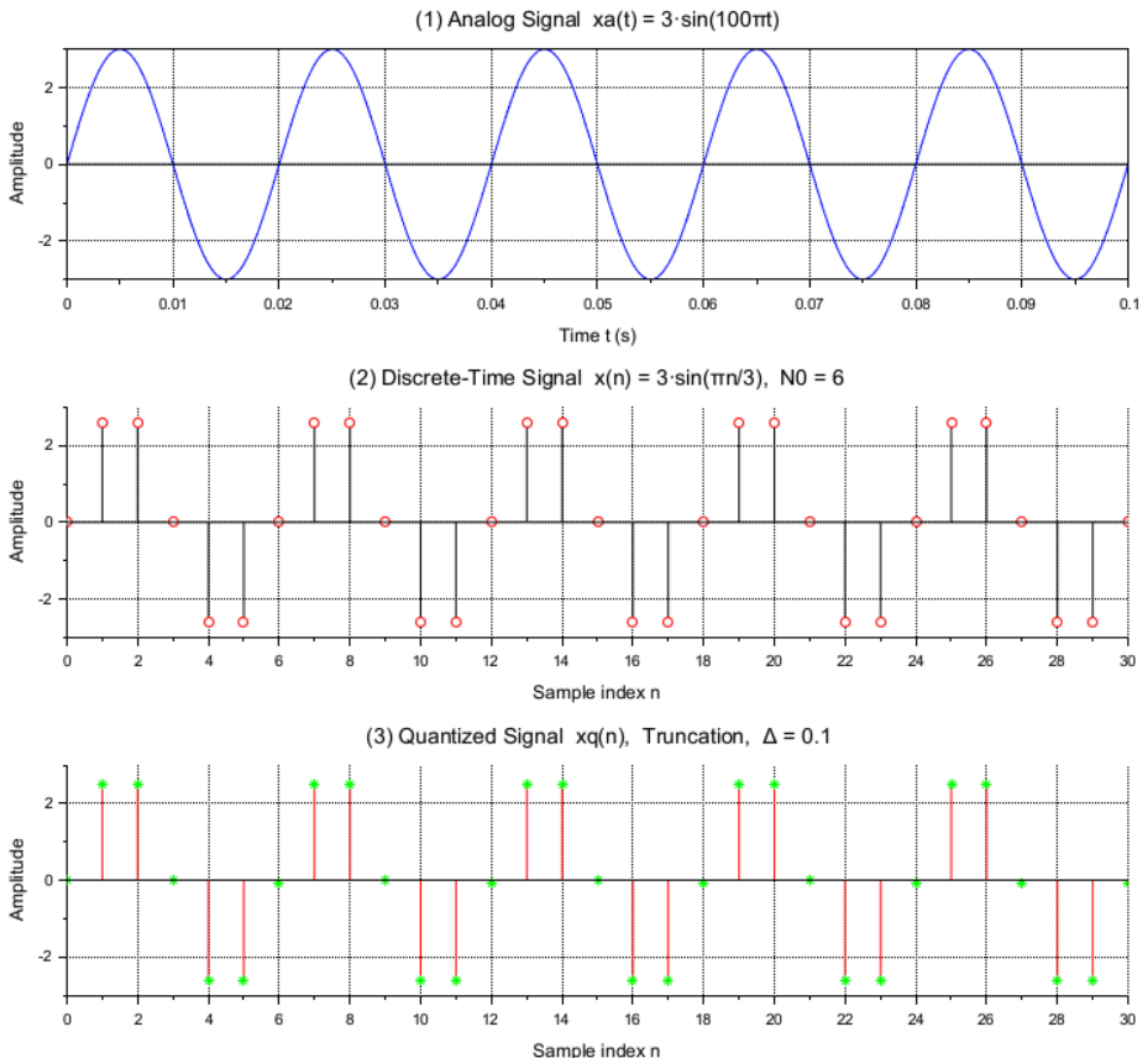
Hình 6: *Đồ thị tín hiệu lượng tử hóa $x_q(n)$ trong 5 chu kỳ*

Phần 5 - Mã lệnh hiển thị tín hiệu

Đoạn mã sau được sử dụng để hiển thị ba tín hiệu: tín hiệu tương tự $x_a(t)$, tín hiệu rời rạc $x(n)$ và tín hiệu lượng tử hóa $x_q(n)$ trong cùng một cửa sổ đồ thị.

```
clf();                              // Clear previous graphics window

// Common parameters
F = 50;            // Signal frequency (Hz)
Fs = 300;          // Sampling frequency (Hz)
T = 1 / F;         // Signal period (s)
Ts = 1 / Fs;       // Sampling period (s)
time = 5 * T;      // Total time: 5 periods
delta = 0.1;       // Quantization step

// 1. Analog Signal xa(t)
t = linspace(0, time, 1000);
xa = 3 * sin(2 * %pi * F * t);

subplot(3, 1, 1);
plot(t, xa, 'b');
plot(t, zeros(t), 'k-');      // Zero reference line (y = 0)
title('(1)Analog Signal xa(t)');
xlabel('Time t (s)');
ylabel('Amplitude');
xgrid();

//  2. Discrete-Time Signal x(n)
// Sampling: x(n) = xa(nTs)
// Periodicity: f = F/Fs = 1/6  -  N0 = 6 samples/cycle
n = 0:30;
xn = 3 * sin(2 * %pi * F * n * Ts);

subplot(3, 1, 2);
plot2d3(n, xn);                         // Stem plot
plot(n, xn, 'ro');                      // Red dots at each sample
plot(n, zeros(n), 'k-');              // Zero reference line (y = 0)
title('(2)Discrete-Time Signal x(n), N0 = 6');
xlabel('Sample index n');
ylabel('Amplitude');
xgrid();

// 3. Quantized Signal xq(n)
xq = floor(xn / delta) * delta;

subplot(3, 1, 3);
plot2d3(n, xq, style=5);               // Stem plot
```

```
plot(n, xq, 'g*');                    // Green stars at each sample
plot(n, zeros(n), 'k-');             // Zero reference line (y = 0)
title('(3) Quantized Signal xq(n), Truncation, delta = 0.1');
xlabel('Sample index n');
ylabel('Amplitude');
xgrid();
```



Hình 7: *Biểu diễn các tín hiệu: $x_a(t)$, $x(n)$ và $x_q(n)$ trong cùng một cửa sổ đồ thị*

Tài liệu

[1] Scilab Enterprises. (2010). *Scilab for beginners*. Orsay, France: Scilab Enterprises. Available at: https://www.scilab.org/sites/default/files/Scilab_beginners.pdf

[2] Scilab Enterprises. (2024). *Scilab — Open source software for numerical computation*. Available at: https://www.scilab.org

[3] Proakis, J. G., & Manolakis, D. G. (2007). *Digital signal processing: Principles, algorithms, and applications* (4th ed.). Upper Saddle River, NJ: Pearson Prentice Hall.

[4] Link Github. (2025). *Signal Processing Exercises using Scilab*. GitHub repository. Available at: https://github.com/anhthu057/XU_LY_TIN_HIEU_SO/tree/lab1

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & THUẬT MÁY TÍNH

XỬ LÝ TÍN HIỆU SỐ (CO2036)

LAB 2

Discrete - Time Signal Analysis and Manipulation Using Scilab

Lớp: L03 - Học Kỳ HK252
Ngày nộp: 12/03/2026

GVHD: Thầy Phạm Công Thái

Sinh Viên	MSSV
Phạm Công Võ	2313946
Nguyễn Anh Thư	2313385

TP.Hồ Chí Minh, Tháng 03/2026

PHÂN CÔNG NHIỆM VỤ VÀ KẾT QUẢ THỰC HIỆN ĐỀ TÀI CỦA TỪNG THÀNH VIÊN NHÓM

STT	Họ và tên	MSSV	Nhiệm vụ	Kết quả	Chữ ký
1	Nguyễn Anh Thư	2313385	• Hiện thực bài tập 2, 4, 6 và 8 • Viết báo cáo	100%	
2	Phạm Công Võ	2313946	• Hiện thực bài tập 1, 3, 5 và 7 • Viết báo cáo	100%	

Mục lục

1 Giới thiệu **4**

2 Mục tiêu bài thực hành **4**

3 Phương pháp hiện thực **4**

4 Tổng quan nội dung **5**

4.1 Các tín hiệu cơ bản 5

4.2 Signal Properties 5

4.3 Simple Signal Manipulations 6

5 Bài Tập Thực Hành **7**

5.1 Exercise 1: Khảo sát các hàm trong Scilab 7

5.2 Exercise 2: Quan sát tín hiệu bước đơn vị 9

5.3 Exercise 3: Quan sát tín hiệu xung đơn vị 11

5.4 Exercise 4: Vẽ tín hiệu Unit Ramp 13

5.5 Exercise 5: Thành phần chẵn và lẻ của tín hiệu 15

5.6 Exercise 6: Cộng hai tín hiệu rời rạc 18

5.7 Exercise 7: Nhân hai tín hiệu rời rạc 21

5.8 Exercise 8: Các phép biến đổi tín hiệu 24

Tài liệu tham khảo **30**

Danh sách hình vẽ

1 Đồ thị tín hiệu bước đơn vị $u(n)$ 10
2 Đồ thị tín hiệu xung đơn vị $\delta(n)$ 12
3 Đồ thị tín hiệu ramp đơn vị $u_r(n)$ 14
4 Đồ thị tín hiệu $x(n)$ và các thành phần chẵn $x_e(n)$ và lẻ $x_o(n)$ 17
5 Đồ thị các tín hiệu $x_1(n)$, $x_2(n)$ và tín hiệu tổng $y(n) = x_1(n) + x_2(n)$ 20
6 Đồ thị các tín hiệu $x_1(n)$, $x_2(n)$ và tín hiệu tích $y(n) = x_1(n)x_2(n)$ 23
7 Đồ thị tín hiệu $x(n)$ và tín hiệu sau phép lật thời gian $y_1(n) = x(-n)$ 28
8 Đồ thị tín hiệu $x(n)$ và tín hiệu sau phép dịch thời gian $y_2(n) = x(n+3)$ 28
9 Đồ thị tín hiệu $x(n)$ và tín hiệu sau phép biến đổi $y_3(n) = 2x(-n-2)$ 29

1 Giới thiệu

Trong lĩnh vực **Xử lý tín hiệu số (Digital Signal Processing - DSP)**, tín hiệu rời rạc theo thời gian đóng vai trò nền tảng trong việc mô hình hóa, phân tích và xử lý các hệ thống tín hiệu số. Các khái niệm như tín hiệu xung đơn vị (unit impulse), tín hiệu bước đơn vị (unit step) hay tín hiệu dốc (unit ramp) được xem là những thành phần cơ bản để xây dựng và biểu diễn nhiều loại tín hiệu phức tạp hơn trong thực tế.

Bên cạnh việc nghiên cứu dạng biểu diễn của tín hiệu, việc phân tích các thuộc tính của tín hiệu như tính tuần hoàn, năng lượng và công suất, cũng như việc thực hiện các phép biến đổi trên tín hiệu (dịch thời gian, gập tín hiệu, thay đổi biên độ, cộng và nhân tín hiệu) là những nội dung quan trọng giúp sinh viên hiểu rõ hơn bản chất của quá trình xử lý tín hiệu số.

Trong bài thực hành này, phần mềm **Scilab** được sử dụng như một công cụ tính toán số nhằm mô phỏng và trực quan hóa các tín hiệu rời rạc. Thông qua việc xây dựng các đoạn chương trình và quan sát kết quả đồ thị, sinh viên có thể liên hệ trực tiếp giữa lý thuyết đã học và hành vi thực tế của tín hiệu trong môi trường tính toán. Qua đó, bài thực hành góp phần nâng cao khả năng phân tích, mô phỏng và đánh giá các đặc tính của tín hiệu rời rạc trong các hệ thống xử lý tín hiệu số.

2 Mục tiêu bài thực hành

- Củng cố kiến thức về tín hiệu rời rạc theo thời gian trong xử lý tín hiệu số.
- Tìm hiểu và mô phỏng các tín hiệu cơ bản như unit impulse, unit step và unit ramp.
- Phân tích một số thuộc tính của tín hiệu như tính tuần hoàn, năng lượng và công suất trung bình.
- Thực hiện các phép biến đổi tín hiệu như dịch thời gian, gập tín hiệu, thay đổi biên độ, cộng và nhân tín hiệu.
- Sử dụng **Scilab** để mô phỏng và trực quan hóa tín hiệu thông qua các đồ thị.

3 Phương pháp hiện thực

Bài thực hành được thực hiện bằng cách kết hợp giữa nghiên cứu lý thuyết và mô phỏng trên phần mềm **Scilab**. Các bước thực hiện chính bao gồm:

1. **Ôn tập lý thuyết** Nghiên cứu các khái niệm về tín hiệu rời rạc, các dạng tín hiệu cơ bản và các phép biến đổi tín hiệu trong xử lý tín hiệu số.
2. **Xây dựng tín hiệu** Thiết lập các tín hiệu rời rạc trong Scilab dựa trên các biểu thức toán học và dãy chỉ số thời gian.
3. **Mô phỏng và trực quan hóa** Triển khai các phép biến đổi tín hiệu bằng mã lệnh Scilab và biểu diễn kết quả bằng đồ thị.
4. **Phân tích kết quả** Quan sát và so sánh kết quả mô phỏng với lý thuyết để đánh giá đặc tính của tín hiệu.
5. **Tổng hợp báo cáo** Tổng hợp hình ảnh kết quả, mô tả và đưa ra nhận xét trong báo cáo thực hành.

4 Tổng quan nội dung

4.1 Các tín hiệu cơ bản

- **Unit Sample Sequence (Impulse)**

  Tín hiệu xung đơn vị, ký hiệu là $\delta(n)$, chỉ có giá trị khác 0 tại thời điểm $n = 0$ và bằng 1 tại điểm này. Ở tất cả các thời điểm khác, giá trị của tín hiệu bằng 0.

$$\delta(n) = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases}$$

  Tín hiệu này thường được sử dụng để phân tích và mô tả đặc tính của hệ thống rời rạc.

- **Unit Step Signal**

  Tín hiệu bước đơn vị, ký hiệu $u(n)$, có giá trị bằng 1 khi $n \geq 0$ và bằng 0 khi $n < 0$.

$$u(n) = \begin{cases} 1, & n \geq 0 \\ 0, & n < 0 \end{cases}$$

  Tín hiệu này được dùng để biểu diễn sự xuất hiện của một tín hiệu tại một thời điểm xác định.

- **Unit Ramp Signal**

  Tín hiệu dốc đơn vị, ký hiệu $u_r(n)$, là tín hiệu tăng tuyến tính theo chỉ số thời gian khi $n \geq 0$.

$$u_r(n) = \begin{cases} n, & n \geq 0 \\ 0, & n < 0 \end{cases}$$

  Tín hiệu ramp thường được sử dụng để mô tả các quá trình tăng tuyến tính theo thời gian.

4.2 Signal Properties

- **Periodic Signal**

  Một tín hiệu $x(n)$ được gọi là tín hiệu tuần hoàn với chu kỳ N ($N > 0$) nếu tồn tại một số nguyên dương N sao cho:

$$x(n + N) = x(n), \quad \forall n$$

  Chu kỳ N nhỏ nhất thỏa mãn điều kiện trên được gọi là chu kỳ cơ bản của tín hiệu.

- **Signal Energy**

  Năng lượng của tín hiệu rời rạc $x(n)$ được định nghĩa là tổng bình phương giá trị của tín hiệu trên toàn bộ miền thời gian:

$$\boldsymbol{E_x = \sum_{n=-\infty}^{\infty} |x(n)|^2}$$

  Đại lượng này phản ánh mức năng lượng tổng thể của tín hiệu.

- **Signal Average Power**

  Công suất trung bình của tín hiệu được định nghĩa là:

$$P_x = \lim_{N\to\infty} \frac{1}{2N+1} \sum_{n=-N}^{N} |x(n)|^2$$

- **Signal Symmetry**

  Một tín hiệu bất kỳ $x(n)$ có thể được biểu diễn dưới dạng tổng của hai thành phần: thành phần chẵn và thành phần lẻ.

$$x(n) = x_e(n) + x_o(n)$$

  Even signal component:

$$x_e(n) = \frac{1}{2}[x(n) + x(-n)]$$

  Odd signal component:

$$x_o(n) = \frac{1}{2}[x(n) - x(-n)]$$

4.3 Simple Signal Manipulations

Các phép biến đổi tín hiệu dùng để thay đổi vị trí hoặc biên độ của tín hiệu trong miền thời gian:

- **Delay:** Tín hiệu bị trễ k đơn vị thời gian được biểu diễn bởi:

$$y(n) = x(n-k), \quad k > 0$$

- **Advance:** Tín hiệu được dịch tiến k đơn vị thời gian:

$$y(n) = x(n+k), \quad k > 0$$

- **Folding:** Phép gập tín hiệu qua trục $n = 0$:

$$y(n) = x(-n)$$

- **Addition:** Phép cộng hai tín hiệu rời rạc:

$$y(n) = x_1(n) + x_2(n)$$

- **Multiplication:** Phép nhân hai tín hiệu rời rạc:

$$y(n) = x_1(n) \cdot x_2(n)$$

- **Amplitude Scaling:** Phép thay đổi biên độ của tín hiệu, trong đó a là hệ số tỉ lệ biên độ.

$$y(n) = a\,x(n)$$

5 Bài Tập Thực Hành

5.1 Exercise 1: Khảo sát các hàm trong Scilab

Yêu cầu đề bài

- **Khảo sát các hàm sau trong Scilab và trình bày ngắn gọn chức năng cũng như cách sử dụng của từng hàm:**
  - `plot2d3(...)`
  - `min(...)`
  - `max(...)`
  - `subplot(...)`
  - `title(...)`
  - `xlabel(...)`
  - `ylabel(...)`
  - `bool2s(...)`
  - `deff(...)`
- **Sinh viên cần mô tả ý nghĩa, chức năng và ví dụ sử dụng cơ bản của từng hàm.**

Bảng 2: *Các hàm Scilab sử dụng trong bài thực hành*

Hàm	Chức năng	Cú pháp & Ví dụ
`plot2d3(x,y)`	Hàm dùng để biểu diễn tín hiệu rời rạc dưới dạng các cột đứng (stem plot). Phương pháp này thường được sử dụng trong xử lý tín hiệu số nhằm minh họa các mẫu tín hiệu theo chỉ số thời gian rời rạc một cách trực quan.	`plot2d3(n, signal)` `n`: vector chỉ số thời gian `signal`: vector biên độ
`min(x)`	Trả về giá trị nhỏ nhất của một vector hoặc ma trận. Khi đầu vào là ma trận, hàm sẽ trả về một vector hàng chứa giá trị nhỏ nhất của từng cột.	`min([5,1,3])` $\rightarrow$ `1`

Hàm	Chức năng	Cú pháp & Ví dụ
`max(x)`	Trả về giá trị lớn nhất của một vector hoặc ma trận. Trong trường hợp đầu vào là ma trận, kết quả trả về là một vector hàng chứa giá trị lớn nhất của từng cột.	`max([5,1,3])` $\rightarrow$ 5
`subplot(m,n,p)`	Chia cửa sổ hiển thị đồ thị thành một lưới gồm m hàng và n cột. Sau đó kích hoạt vị trí p để vẽ đồ thị tương ứng, cho phép hiển thị nhiều tín hiệu trong cùng một cửa sổ.	`subplot(3,1,2)` Chia cửa sổ thành 3 hàng, 1 cột và chọn ô thứ 2
`title('text')`	Thêm tiêu đề mô tả cho đồ thị đang hiển thị. Tiêu đề giúp người đọc dễ dàng nhận biết nội dung và loại tín hiệu được biểu diễn.	`title('Unit Step Signal u(n)')`
`xlabel('text')`	Thêm nhãn cho trục hoành (x) của đồ thị. Nhãn trục thường được sử dụng để mô tả chỉ số mẫu hoặc biến thời gian của tín hiệu.	`xlabel('n (samples)')`
`ylabel('text')`	Thêm nhãn cho trục tung (y) nhằm mô tả giá trị biên độ của tín hiệu hiển thị trên đồ thị.	`ylabel('Amplitude')`
`bool2s(expr)`	Chuyển đổi biểu thức logic TRUE / FALSE thành giá trị số tương ứng 1 hoặc 0. Hàm này thường được sử dụng để xây dựng các tín hiệu điều kiện như tín hiệu bước hoặc xung đơn vị.	`bool2s(n >= 0)` 1 nếu $n \geq 0$, ngược lại 0
`deff('y=f(x)','body')`	Cho phép định nghĩa một hàm inline trực tiếp trong môi trường Scilab mà không cần tạo file hàm riêng. Điều này giúp việc xây dựng và thử nghiệm các hàm toán học trở nên nhanh chóng và thuận tiện.	`deff('y=sq(x)','y=x.^2')` Định nghĩa hàm $f(x) = x^2$

5.2 Exercise 2: Quan sát tín hiệu bước đơn vị

Yêu cầu đề bài

- **Chạy các lệnh sau trong Scilab:**

```
n = -5:5;
msignal = bool2s (n >= 0);
plot2d3(n, msignal)
```

- **Quan sát kết quả đồ thị thu được và trình bày hiểu biết của bạn về tín hiệu được tạo ra.**
- **Giải thích ý nghĩa của hàm `bool2s()` trong đoạn chương trình trên.**

Script Scilab

```
1 n = -5:5;
2 msignal = bool2s(n >= 0);
3 plot2d3(n, msignal);
```

1. Phân tích chương trình

Dòng 1:

- Lệnh `n = -5:5;` tạo một vector chỉ số thời gian rời rạc gồm 11 phần tử:

$$n = \{-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5\}$$

- Vector này đại diện cho miền thời gian rời rạc được sử dụng để khảo sát tín hiệu.
- Việc lựa chọn khoảng giá trị từ -5 đến 5 giúp quan sát rõ vị trí xuất hiện của xung tại thời điểm $n = 0$.

Dòng 2:

- Lệnh `msignal = bool2s(n >= 0);` tạo tín hiệu bước đơn vị dựa trên điều kiện logic $n \geq 0$.
- Biểu thức `(n >= 0)` tạo ra một vector giá trị logic (TRUE/FALSE).
- Hàm `bool2s()` chuyển đổi các giá trị logic này thành giá trị số tương ứng:

$$TRUE \to 1, \quad FALSE \to 0$$

n	-5	-4	-3	-2	-1	0	1	2	3	4	5
$n \geq 0$	F	F	F	F	F	T	T	T	T	T	T
$u(n)$	0	0	0	0	0	1	1	1	1	1	1

Dòng 3:

- Lệnh `plot2d3(n, msignal);` được sử dụng để biểu diễn tín hiệu rời rạc dưới dạng các cột đứng (stem plot).
- Đồ thị thu được minh họa sự thay đổi của tín hiệu bước đơn vị theo chỉ số thời gian n.

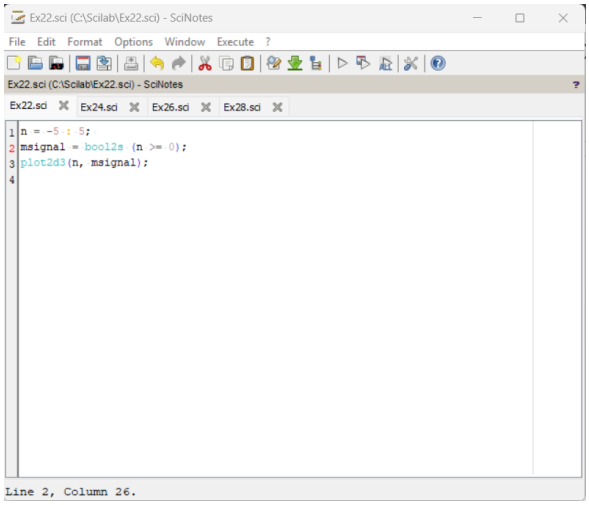
2. Định nghĩa toán học

Tín hiệu Bước đơn vị (Unit Step Signal). trong miền thời gian rời rạc được định nghĩa như sau:

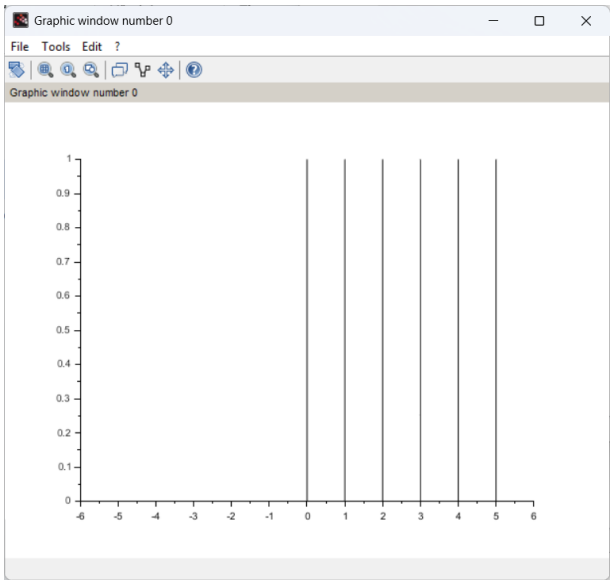
$$u(n) = \begin{cases} 1, & \text{khi } n \geq 0 \\ 0, & \text{khi } n < 0 \end{cases}$$

3. Nhận xét

- Tín hiệu Bước đơn vị $u(n)$ là một trong những tín hiệu cơ bản và quan trọng nhất trong lĩnh vực xử lý tín hiệu số.
- Tín hiệu này có giá trị bằng 0 đối với mọi $n < 0$ và chuyển sang giá trị 1 kể từ thời điểm $n = 0$.
- Đồ thị của tín hiệu thể hiện rõ sự thay đổi đột ngột tại $n = 0$, với các mẫu tín hiệu trước đó bằng 0 và các mẫu sau đó bằng 1.
- Trong thực tế, tín hiệu bước đơn vị thường được sử dụng để mô tả quá trình khởi động của một hệ thống hoặc biểu diễn phản ứng của hệ thống khi xuất hiện một tác động đột ngột tại một thời điểm xác định.



(a) Scilab Script



Hình 1: *Đồ thị tín hiệu bước đơn vị $u(n)$*

5.3 Exercise 3: Quan sát tín hiệu xung đơn vị

Yêu cầu đề bài

- **Chạy các lệnh sau trong Scilab:**

```
n = -5:5;
msignal = bool2s (n == 0);
plot2d3(n, msignal)
```

- **Quan sát đồ thị đầu ra và trình bày nhận xét của bạn về tín hiệu thu được.**
- **Giải thích ý nghĩa của điều kiện `(n == 0)` trong việc tạo tín hiệu.**

Script Scilab

```
1 n = -5:5;
2 msignal = bool2s(n == 0);
3 plot2d3(n, msignal);
```

1. Phân tích chương trình

Dòng 1:

- Lệnh `n = -5:5;` tạo một vector chỉ số thời gian rời rạc gồm 11 phần tử:

$$n = \{-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5\}$$

- Vector này đại diện cho miền thời gian rời rạc được sử dụng để khảo sát tín hiệu trong thí nghiệm.
- Việc lựa chọn khoảng giá trị từ -5 đến 5 giúp quan sát rõ vị trí xuất hiện của xung tại thời điểm $n = 0$.

Dòng 2:

- Lệnh `msignal = bool2s(n == 0);` được sử dụng để tạo tín hiệu xung đơn vị.
- Biểu thức logic `(n == 0)` kiểm tra điều kiện từng phần tử của vector n:
  - Nếu $n = 0 \rightarrow$ giá trị logic TRUE
  - Nếu $n \neq 0 \rightarrow$ giá trị logic FALSE
- Hàm `bool2s()` chuyển đổi các giá trị logic này thành giá trị số tương ứng:

$$TRUE \rightarrow 1, \quad FALSE \rightarrow 0$$

n	-5	-4	-3	-2	-1	0	1	2	3	4	5
$n = 0$	F	F	F	F	F	T	F	F	F	F	F
$\delta(n)$	0	0	0	0	0	1	0	0	0	0	0

Dòng 3:

- Lệnh `plot2d3(n, msignal);` được sử dụng để vẽ đồ thị tín hiệu rời rạc.
- Hàm này biểu diễn mỗi mẫu tín hiệu bằng một cột đứng (stem plot), giúp minh họa trực quan giá trị của tín hiệu tại từng thời điểm n.
- Kết quả đồ thị cho thấy chỉ tồn tại một cột duy nhất có biên độ bằng 1 tại $n = 0$, trong khi tất cả các vị trí khác đều có giá trị bằng 0.

2. Định nghĩa toán học

Tín hiệu Xung đơn vị (Unit Impulse Signal được định nghĩa trong miền thời gian rời rạc như sau:

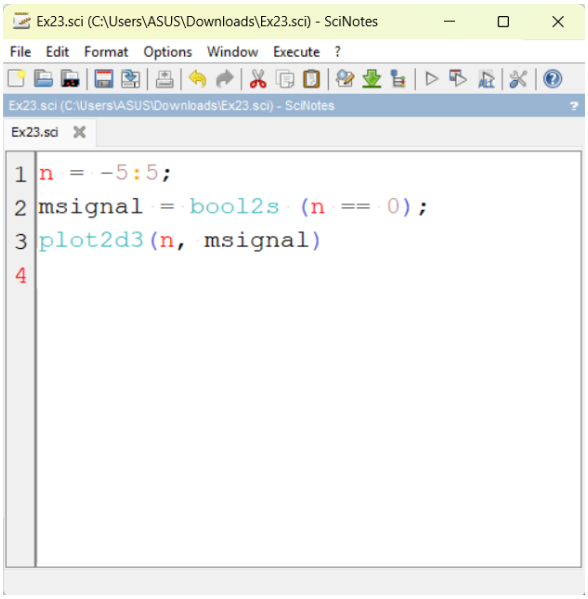
$$\delta(n) = \begin{cases} 1, & \text{khi } n = 0 \\ 0, & \text{khi } n \neq 0 \end{cases}$$

Tín hiệu này chỉ tồn tại tại một thời điểm duy nhất $n = 0$ và bằng 0 ở tất cả các thời điểm còn lại.

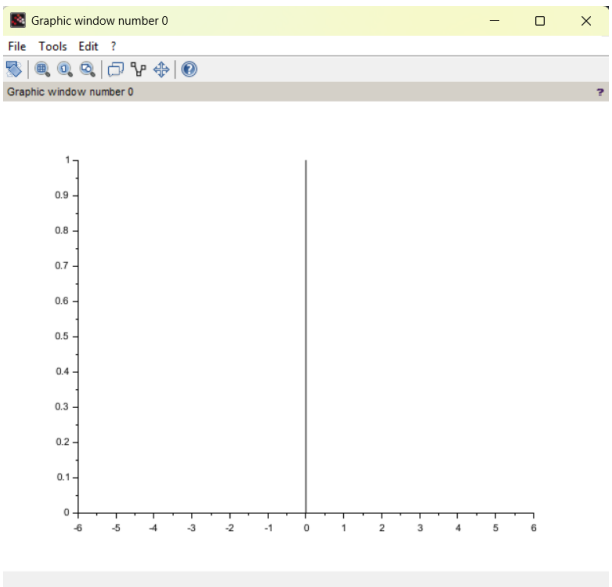
3. Nhận xét

- Tín hiệu xung đơn vị $\delta(n)$ là một trong những tín hiệu cơ bản trong lý thuyết xử lý tín hiệu số. Tín hiệu chỉ có giá trị bằng 1 tại thời điểm $n = 0$ và bằng 0 tại tất cả các thời điểm khác.
- Do đặc tính chỉ tồn tại tại một thời điểm duy nhất, tín hiệu xung đơn vị thường được sử dụng để mô tả các tác động tức thời hoặc các kích thích rất ngắn trong hệ thống.
- Ngoài ra, tín hiệu bước đơn vị cũng có mối liên hệ chặt chẽ với tín hiệu xung đơn vị thông qua tổng tích lũy:

$$u(n) = \sum_{k=-\infty}^{n} \delta(k)$$



(a) Scilab Script



Hình 2: *Đồ thị tín hiệu xung đơn vị $\delta(n)$*

5.4 Exercise 4: Vẽ tín hiệu Unit Ramp

Yêu cầu đề bài

- **Sử dụng Scilab để xây dựng và vẽ tín hiệu unit ramp $u_r(n)$.**
- **Phạm vi chỉ số thời gian được xét là:**

$$n = -5 : 5$$

- **Biểu diễn tín hiệu dưới dạng đồ thị bằng các hàm vẽ trong Scilab.**

1. Định nghĩa toán học

Tín hiệu Ramp đơn vị (Unit Ramp Signal) trong miền thời gian rời rạc được định nghĩa như sau:

$$ur(n) = \begin{cases} n, & \text{khi } n \geq 0 \\ 0, & \text{khi } n < 0 \end{cases}$$

Tín hiệu này có thể được biểu diễn thông qua tín hiệu bước đơn vị:

$$ur(n) = n \cdot u(n)$$

trong đó $u(n)$ là tín hiệu bước đơn vị.

2. Ý tưởng

- Tín hiệu ramp có giá trị bằng n khi $n \geq 0$ và bằng 0 khi $n < 0$.
- Do đó ta có thể xây dựng tín hiệu này bằng cách nhân vector n với tín hiệu bước đơn vị $u(n)$.
- Trong Scilab, tín hiệu bước được tạo bằng:

$$u(n) = \texttt{bool2s}(n \geq 0)$$

- Vì vậy tín hiệu ramp có thể viết dưới dạng:

$$ur(n) = n \times u(n) = n \times \texttt{bool2s}(n \geq 0)$$

3. Script Scilab

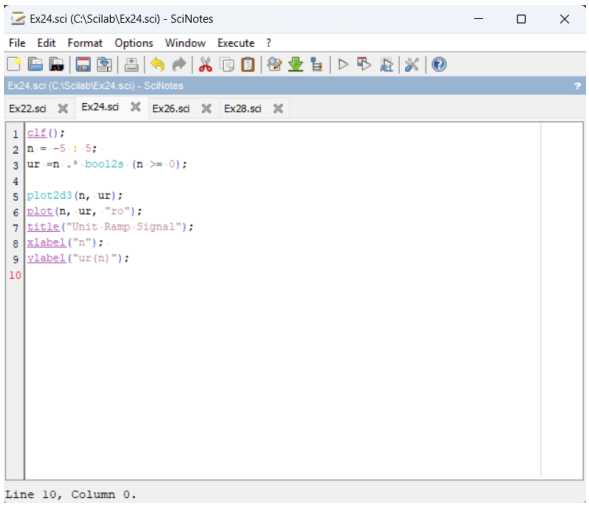
```
n = -5:5;
ur = n .* bool2s(n >= 0);
plot2d3(n, ur);
plot(n, ur, "ro");
title("Unit Ramp Signal");
xlabel("n");
ylabel("ur(n)");
```

4. Bảng kết quả tính tay

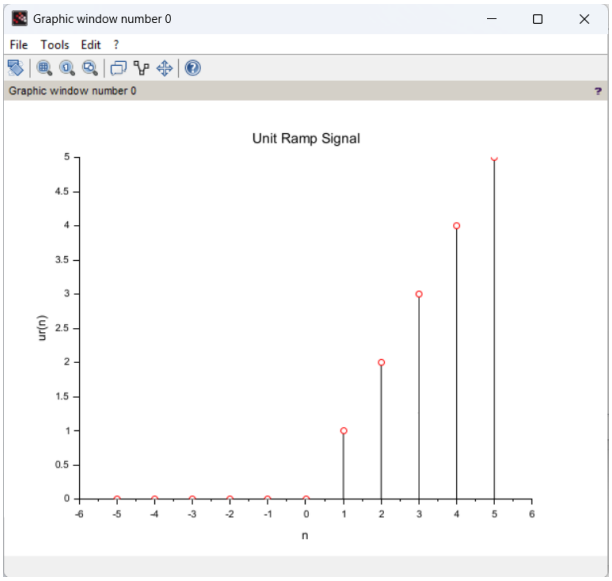
n	-5	-4	-3	-2	-1	0	1	2	3	4	5
$u(n)$	0	0	0	0	0	1	1	1	1	1	1
$ur(n)$	0	0	0	0	0	0	1	2	3	4	5

5. Nhận xét

- Tín hiệu Ramp đơn vị $ur(n)$ được tạo ra bằng phép nhân từng phần tử giữa vector chỉ số thời gian n và tín hiệu bước đơn vị $u(n)$.
- Đối với mọi $n < 0$, giá trị của tín hiệu bằng 0. Khi $n \geq 0$, giá trị của tín hiệu tăng tuyến tính theo n.
- Đồ thị thu được thể hiện dạng đường thẳng tăng dần bắt đầu từ $n = 0$, trong khi toàn bộ phần bên trái bằng 0.
- Tín hiệu ramp thường được sử dụng trong phân tích hệ thống để mô tả các tín hiệu tăng tuyến tính theo thời gian, ví dụ như quá trình tăng tốc của hệ thống hoặc tín hiệu điều khiển tăng dần.



(a) Scilab Script



Hình 3: *Đồ thị tín hiệu ramp đơn vị $u_r(n)$*

5.5 Exercise 5: Thành phần chẵn và lẻ của tín hiệu

Yêu cầu đề bài

- **Cho tín hiệu rời rạc:**

$$x(n) = \{1, 3\uparrow, -2\}$$

- **Sử dụng Scilab để vẽ:**
  - **Tín hiệu gốc** $x(n)$
  - **Thành phần chẵn** $x_e(n)$
  - **Thành phần lẻ** $x_o(n)$
- **Mỗi tín hiệu được vẽ bằng một đồ thị riêng nhưng tất cả các đồ thị phải được hiển thị trong cùng một cửa sổ.**
- **Sử dụng các hàm `title()`, `xlabel()` và `ylabel()` để hiển thị tên và nhãn cho từng đồ thị.**

1. Xác định tín hiệu gốc

Cho tín hiệu rời rạc:

$$x(n) = \{1, 3\uparrow, -2\}$$

Ký hiệu $\uparrow$ chỉ vị trí gốc $n = 0$. Khi đó ta có:

$$x(-1) = 1, \qquad x(0) = 3, \qquad x(1) = -2$$

Do đó vector chỉ số thời gian tương ứng:

$$n = \{-1, 0, 1\}$$

2. Công thức phân tích chẵn và lẻ

Mọi tín hiệu rời rạc $x(n)$ đều có thể được phân tích thành tổng của hai thành phần:

- **Thành phần chẵn (Even component)**

$$x_e(n) = \frac{x(n) + x(-n)}{2}$$

Thành phần này có tính chất:

$$x_e(-n) = x_e(n)$$

– **Thành phần lẻ (Odd component)**

$$x_o(n) = \frac{x(n) - x(-n)}{2}$$

Thành phần này thỏa mãn:

$$x_o(-n) = -x_o(n)$$

Tổng của hai thành phần sẽ khôi phục lại tín hiệu ban đầu:

$$x(n) = x_e(n) + x_o(n)$$

3. Tính toán chi tiết

Đầu tiên xác định tín hiệu đảo thời gian $x(-n)$:

$$x(-n) = \{-2,\ 3\uparrow,\ 1\}$$

n	$x(n)$	$x(-n)$	$x_e(n)$	$x_o(n)$
-1	1	-2	$(1+(-2))/2 = -0.5$	$(1-(-2))/2 = 1.5$
0	3	3	$(3+3)/2 = 3$	$(3-3)/2 = 0$
1	-2	1	$(-2+1)/2 = -0.5$	$(-2-1)/2 = -1.5$

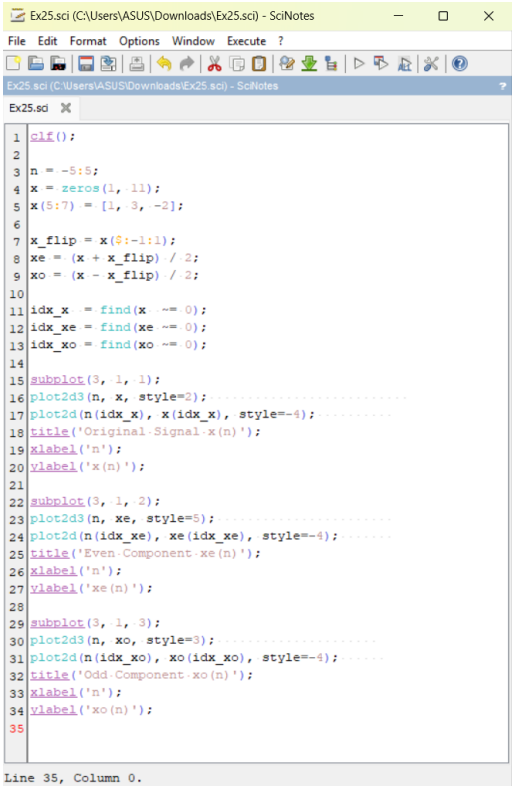
Do đó ta thu được:

$$x_e(n) = \{-0.5,\ 3\uparrow,\ -0.5\}$$

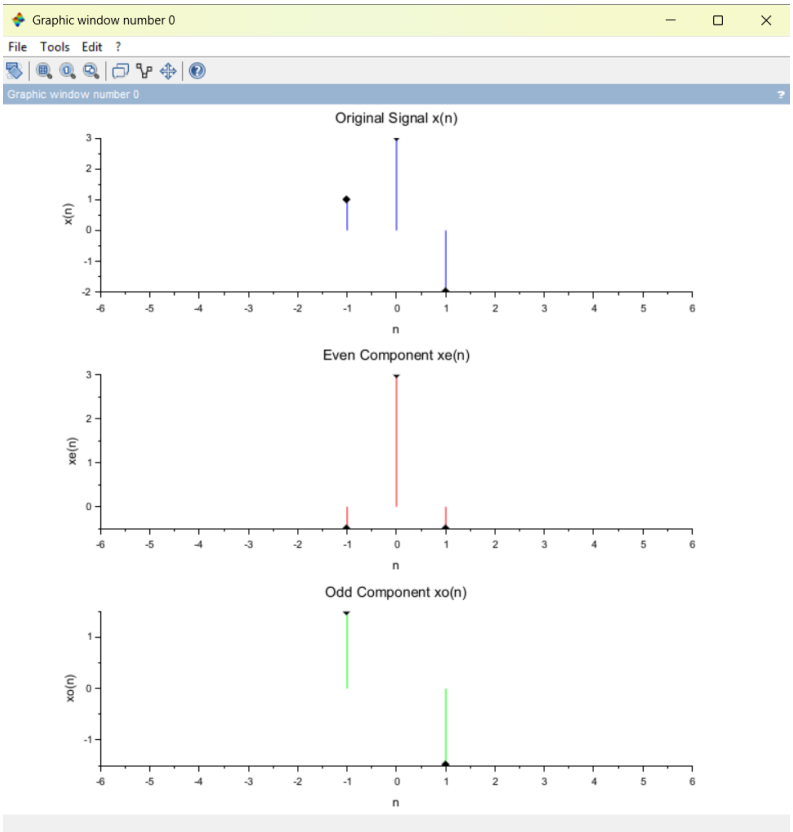
$$x_o(n) = \{1.5,\ 0\uparrow,\ -1.5\}$$

Kiểm tra lại:

$$x_e(n) + x_o(n) = [-0.5,\ 3,\ -0.5] + [1.5,\ 0,\ -1.5] = [1,\ 3,\ -2] = x(n)$$



a/ Scilab Script



Hình 4: *Đồ thị tín hiệu $x(n)$ và các thành phần chẵn $x_e(n)$ và lẻ $x_o(n)$*

5.6 Exercise 6: Cộng hai tín hiệu rời rạc

Yêu cầu đề bài

- **Cho hai tín hiệu rời rạc:**

$$x_1(n) = \{0\uparrow, 1, 3, -2\}$$

$$x_2(n) = \{0, 1\uparrow, 2, 3\}$$

- **Xác định tín hiệu:**

$$y(n) = x_1(n) + x_2(n)$$

- **Sử dụng Scilab để vẽ các tín hiệu $x_1(n)$, $x_2(n)$ và $y(n)$.**
- **Mỗi tín hiệu được vẽ trên một đồ thị riêng nhưng hiển thị trong cùng một cửa sổ.**
- **Sử dụng `title()`, `xlabel()` và `ylabel()` để đặt tên cho từng đồ thị.**

1. Xác định hai tín hiệu

Cho hai tín hiệu rời rạc:

$$x_1(n) = \{0\uparrow, 1, 3, -2\}$$

$$x_2(n) = \{0, 1\uparrow, 2, 3\}$$

Trong đó ký hiệu $\uparrow$ biểu thị vị trí của $n = 0$.

Do đó ta xác định được miền tồn tại của các tín hiệu:

Tín hiệu	Giá trị	Miền tồn tại
$x_1(n)$	$x_1(0) = 0$, $x_1(1) = 1$, $x_1(2) = 3$, $x_1(3) = -2$	$n_1 : 0 \to 3$
$x_2(n)$	$x_2(-1) = 0$, $x_2(0) = 1$, $x_2(1) = 2$, $x_2(2) = 3$	$n_2 : -1 \to 2$

2. Nguyên tắc cộng tín hiệu (Zero Padding)

Để thực hiện phép cộng hai tín hiệu có miền thời gian khác nhau, cần thực hiện các bước sau:

- Xác định miền thời gian chung của hai tín hiệu:

$$n_{start} = \min(0, -1) = -1, \qquad n_{end} = \max(3, 2) = 3$$

- Mở rộng (zero padding) hai tín hiệu về cùng miền thời gian bằng cách thêm giá trị 0 tại các vị trí không xác định.
- Thực hiện phép cộng từng phần tử:

$$y(n) = x_1(n) + x_2(n)$$

3. Biểu diễn hai tín hiệu trên miền chung

Sau khi mở rộng miền thời gian, ta có:

$$x_1(n) = \{0, 0\uparrow, 1, 3, -2\}$$

$$x_2(n) = \{0, 1\uparrow, 2, 3, 0\}$$

với miền thời gian chung:

$$n = \{-1, 0, 1, 2, 3\}$$

4. Bảng tính tay

n	-1	0	1	2	3
$x_1(n)$	0	0	1	3	-2
$x_2(n)$	0	1	2	3	0
$y(n) = x_1(n) + x_2(n)$	0	1	3	6	-2

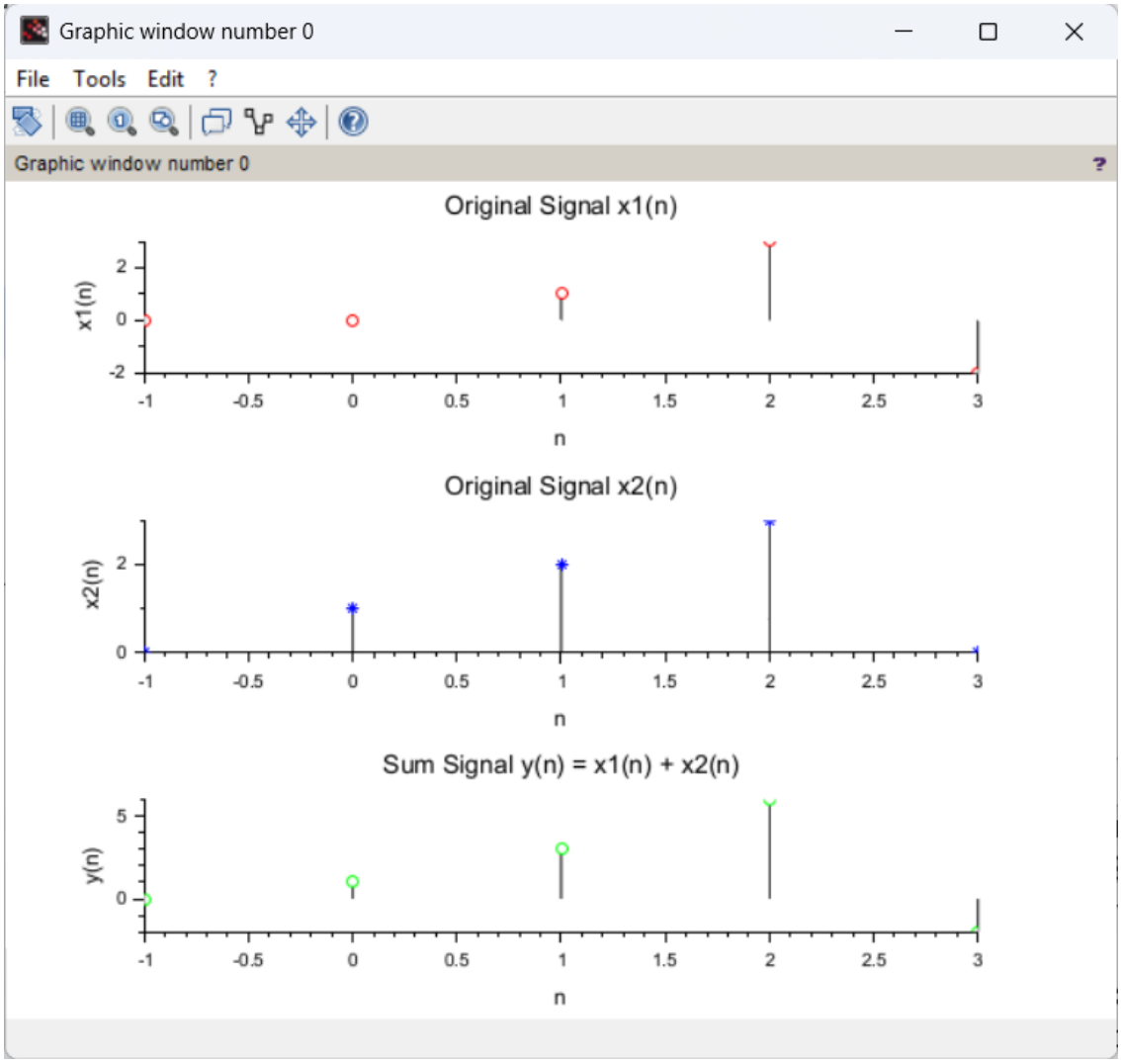
Do đó ta thu được tín hiệu kết quả:

$$y(n) = \{0, 1\uparrow, 3, 6, -2\}$$

5. Script Scilab

```
n = -1 : 3;
x1 = [0, 0, 1, 3, -2];
x2 = [0, 1, 2, 3, 0];
y = x1 + x2;

subplot(3, 1, 1);
plot2d3(n, x1);
plot(n, x1, "ro");
title("Original Signal x1(n)");
xlabel("n");
ylabel("x1(n)");
```

```
subplot(3, 1, 2);
plot2d3(n, x2);
plot(n, x2, "b*");
title("Original Signal x2(n)");
xlabel("n");
ylabel("x2(n)");

subplot(3, 1, 3);
plot2d3(n, y);
plot(n, y, "go");
title("Sum Signal y(n) = x1(n) + x2(n)");
xlabel("n");
ylabel("y(n)");
```



Hình 5: *Đồ thị các tín hiệu $x_1(n)$, $x_2(n)$ và tín hiệu tổng $y(n) = x_1(n) + x_2(n)$*

Đồ thị $y(n)$ cho thấy sự thay đổi biên độ tại từng vị trí mẫu là tổng đại số của các mẫu tương ứng từ x_1 và x_2. Kết quả từ đồ thị cũng trùng khớp với kết quả tính toán được ở trên.

5.7 Exercise 7: Nhân hai tín hiệu rời rạc

Yêu cầu đề bài

- **Cho hai tín hiệu rời rạc:**

$$x_1(n) = \{0\uparrow, 1, 3, -2\}$$

$$x_2(n) = \{0, 1\uparrow, 2, 3\}$$

- **Xác định tín hiệu:**

$$y(n) = x_1(n) \cdot x_2(n)$$

- **Sử dụng Scilab để vẽ các tín hiệu $x_1(n)$, $x_2(n)$ và $y(n)$.**
- **Mỗi tín hiệu được biểu diễn bằng một đồ thị riêng và hiển thị trong cùng một cửa sổ.**
- **Sử dụng các hàm `title()`, `xlabel()` và `ylabel()` để đặt nhãn cho từng đồ thị.**

1. Xác định hai tín hiệu

Cho hai tín hiệu rời rạc:

$$x_1(n) = \{0\uparrow, 1, 3, -2\}$$

$$x_2(n) = \{0, 1\uparrow, 2, 3\}$$

Trong đó ký hiệu $\uparrow$ biểu thị vị trí của $n = 0$.

Do đó ta xác định được miền tồn tại của các tín hiệu:

Tín hiệu	Giá trị	Miền tồn tại
$x_1(n)$	$x_1(0) = 0$, $x_1(1) = 1$, $x_1(2) = 3$, $x_1(3) = -2$	$n_1 : 0 \to 3$
$x_2(n)$	$x_2(-1) = 0$, $x_2(0) = 1$, $x_2(1) = 2$, $x_2(2) = 3$	$n_2 : -1 \to 2$

2. Nguyên tắc nhân tín hiệu (Zero Padding)

Để thực hiện phép nhân hai tín hiệu có miền thời gian khác nhau, ta tiến hành:

- Xác định miền thời gian chung của hai tín hiệu:

$$n_{start} = \min(0, -1) = -1, \qquad n_{end} = \max(3, 2) = 3$$

- Mở rộng (zero padding) hai tín hiệu về cùng miền thời gian bằng cách thêm giá trị 0 tại các vị trí không xác định.
- Thực hiện phép nhân từng phần tử:

$$y(n) = x_1(n) \cdot x_2(n)$$

3. Biểu diễn hai tín hiệu trên miền chung

Sau khi mở rộng miền thời gian, ta có:

$$x_1(n) = \{0, 0\uparrow, 1, 3, -2\}$$

$$x_2(n) = \{0, 1\uparrow, 2, 3, 0\}$$

với miền thời gian chung:

$$n = \{-1, 0, 1, 2, 3\}$$

4. Bảng tính tay

n	-1	0	1	2	3
$x_1(n)$	0	0	1	3	-2
$x_2(n)$	0	1	2	3	0
$y(n) = x_1(n) \cdot x_2(n)$	0	0	2	9	0

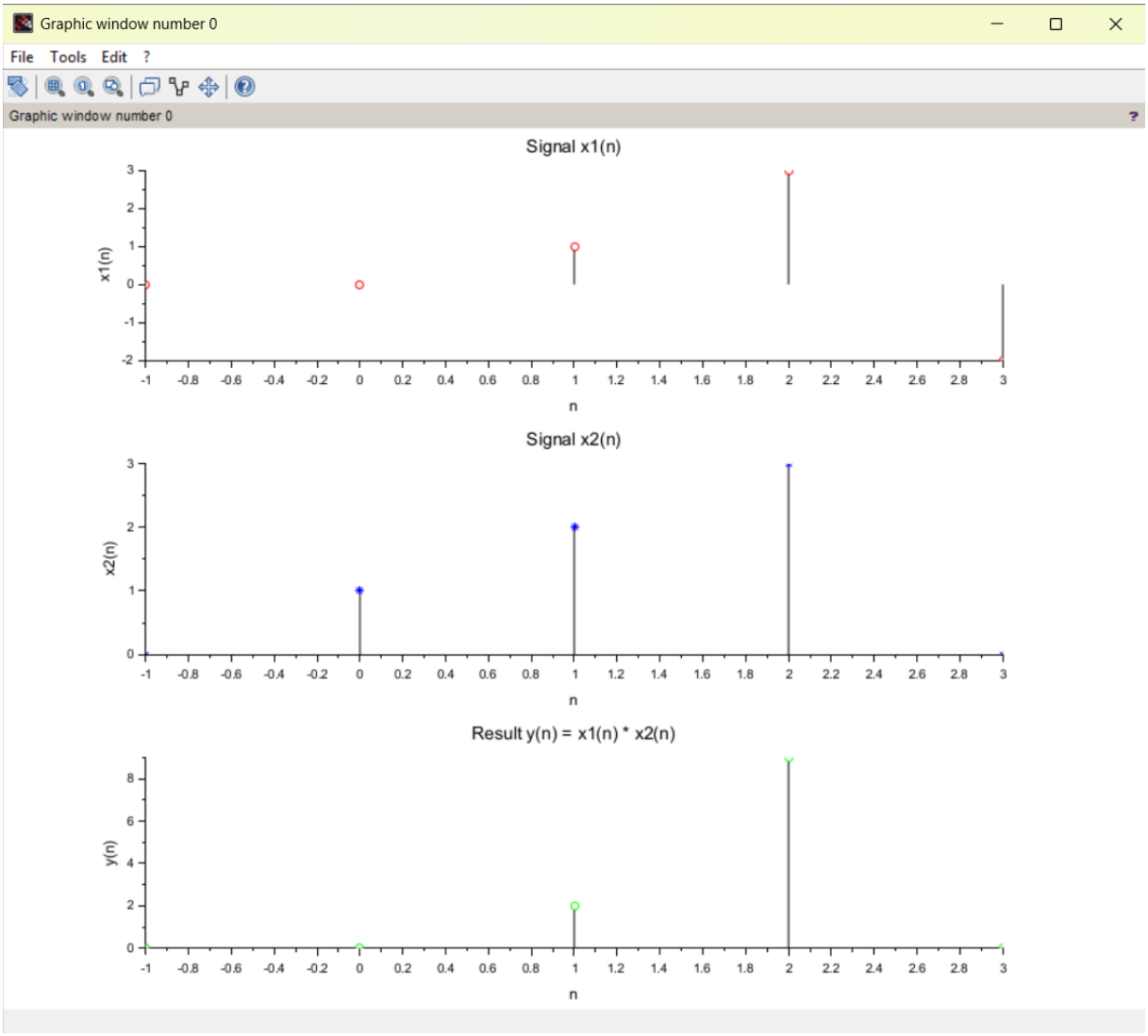
Do đó ta thu được tín hiệu kết quả:

$$y(n) = \{0, 0\uparrow, 2, 9, 0\}$$

5. Script Scilab

```
n = -1:3;
x1 = [0, 0, 1, 3, -2];
x2 = [0, 1, 2, 3, 0];
y = x1 .* x2;

subplot(3,1,1);
plot2d3(n, x1);
plot(n, x1, "ro");
title("Signal x1(n)");
xlabel("n");
ylabel("x1(n)");
```

```
subplot(3,1,2);
plot2d3(n, x2);
plot(n, x2, "b*");
title("Signal x2(n)");
xlabel("n");
ylabel("x2(n)");

subplot(3,1,3);
plot2d3(n, y);
plot(n, y, "go");
title("Result y(n) = x1(n) * x2(n)");
xlabel("n");
ylabel("y(n)");
```



Hình 6: *Đồ thị các tín hiệu $x_1(n)$, $x_2(n)$ và tín hiệu tích $y(n) = x_1(n)x_2(n)$*

5.8 Exercise 8: Các phép biến đổi tín hiệu

Yêu cầu đề bài

- **Cho tín hiệu rời rạc:**

$$x(n) = \{1, -2, 3\uparrow, 6\}$$

- **Xác định các tín hiệu sau:**
  - $y_1(n) = x(-n)$
  - $y_2(n) = x(n+3)$
  - $y_3(n) = 2x(-n-2)$
- **Sử dụng Scilab để vẽ tín hiệu gốc $x(n)$ và các tín hiệu biến đổi $y_i(n)$.**
- **Mỗi cặp tín hiệu (tín hiệu gốc và tín hiệu biến đổi) được hiển thị trong cùng một cửa sổ đồ thị.**
- **Sử dụng các hàm `title()`, `xlabel()` và `ylabel()` để đặt tên và nhãn cho các đồ thị.**

1. Xác định tín hiệu gốc

Cho tín hiệu rời rạc:

$$x(n) = \{1, -2, 3\uparrow, 6\}$$

Trong đó ký hiệu $\uparrow$ biểu thị vị trí của $n = 0$. Do đó ta có bảng giá trị của tín hiệu:

n	-2	-1	0	1
$x(n)$	1	-2	3	6

2. Các phép biến đổi tín hiệu

a) Phép lật tín hiệu (Folding)

Tín hiệu:

$$y_1(n) = x(-n)$$

Nguyên tắc: phép lật thời gian tương đương với việc đảo ngược tín hiệu quanh gốc $n = 0$.

n	-1	0	1	2
$y_1(n)$	6	3	-2	1

Do đó:

$$y_1(n) = \{6, 3\uparrow, -2, 1\}$$

b) Phép dịch Trái tín hiệu (Advance)

Tín hiệu:

$$y_2(n) = x(n+3)$$

Nguyên tắc: khi thay n bằng $(n+3)$, tín hiệu sẽ được dịch sang trái 3 đơn vị trên trục thời gian.

Bảng giá trị:

n	-5	-4	-3	-2	-1	0
$y_2(n)$	1	-2	3	6	0	0

Do đó:

$$y_2(n) = \{1, -2, 3, 6, 0, 0\uparrow\}$$

c) Biến đổi kết hợp: lật, dịch và nhân biên độ

Tín hiệu cần tìm:

$$y_3(n) = 2x(-n-2)$$

Bước 1: Lật tín hiệu

$$x(-n) = \{6, 3\uparrow, -2, 1\}$$

Bước 2: Nhân biên độ

$$2x(-n) = \{12, 6\uparrow, -4, 2\}$$

Bước 3: Dịch trái 2 đơn vị

Thay $n = n+2$ vào biểu thức $2x(-n)$:

$$2x(-(n+2))$$

Suy ra:

$$y_3(n) = 2x(-n-2)$$

Tương đương với việc dịch tín hiệu $2x(-n)$ sang trái (Advance) 2 đơn vị trên trục thời gian.

n	-3	-2	-1	0
$y_3(n)$	12	6	-4	2

Do đó:

$$y_3(n) = \{12, 6, -4, 2 \uparrow\}$$

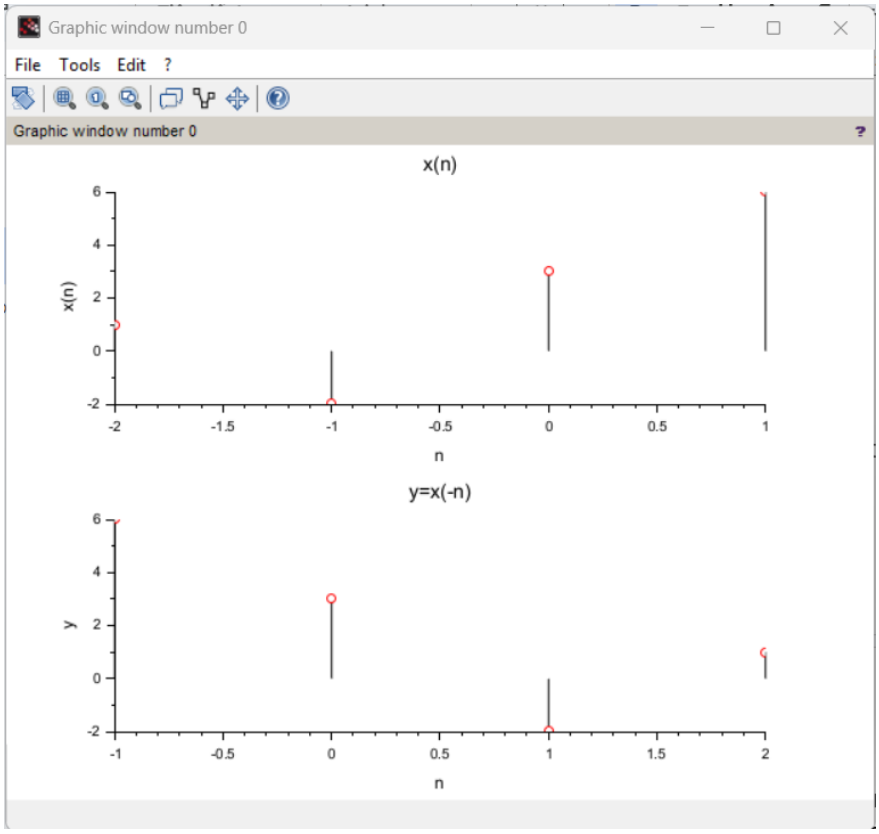
trong đó ký hiệu $\uparrow$ biểu thị vị trí $n = 0$.

3. Script Scilab

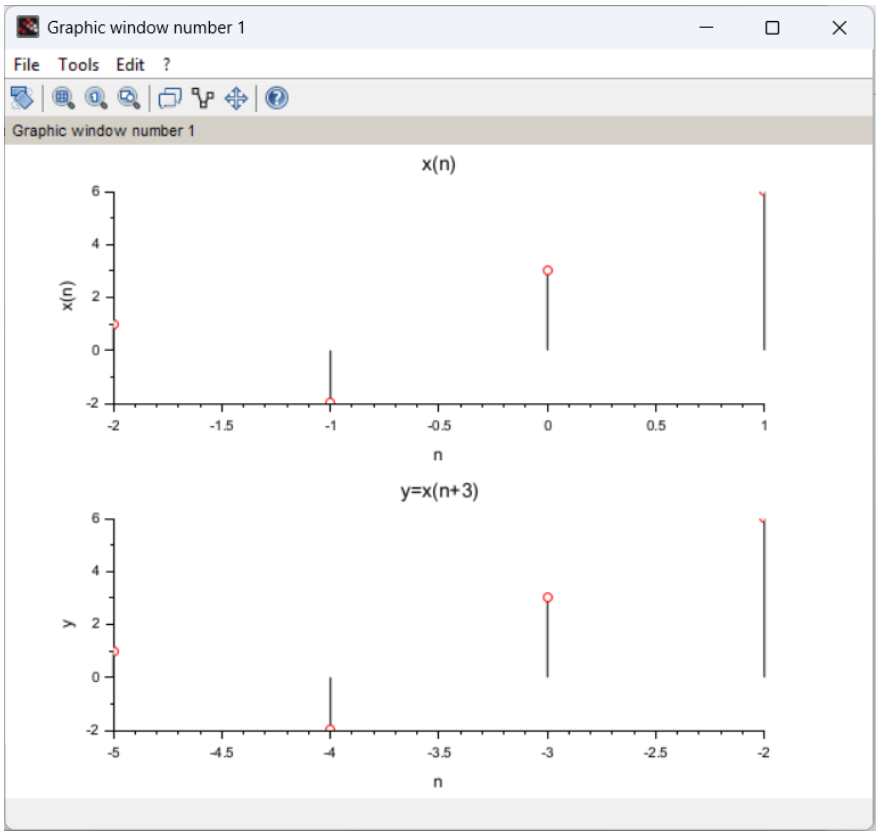
```
clf();
n = -2 : 1;
x = [1, -2, 3, 6];

//cau 8a
n1 =-n($ : -1 : 1);
y1 = x($ : -1 : 1);

subplot(2, 1, 1);
plot2d3(n, x);
plot(n, x, "ro");
title("x(n)");
xlabel("n");
ylabel("x(n)");

subplot(2, 1, 2);
plot2d3(n1, y1);
plot(n1, y1, "ro");
title("y=x(-n)");
xlabel("n");
ylabel("y");

//cau 8b
scf(1);
n2 = n - 3;
y2 = x;

subplot(2, 1, 1);
plot2d3(n, x);
plot(n, x, "ro");
title("x(n)");
xlabel("n");
ylabel("x(n)");
```

```

subplot(2, 1, 2);
plot2d3(n2, y2);
plot(n2, y2, "ro");
title("y=x(n+3)");
xlabel("n");
ylabel("y");

//cau 8c
scf(2);
n3 = -n($:-1:1) - 2;
y3 = 2 * x($:-1:1);

subplot(2, 1, 1);
plot2d3(n, x);
plot(n, x, "ro");
title("x(n)");
xlabel("n");
ylabel("x(n)");

subplot(2, 1, 2);
plot2d3(n3, y3);
plot(n3, y3, "ro");
title("y(n)=2x(-n-2)");
xlabel("n");
ylabel("y");
```

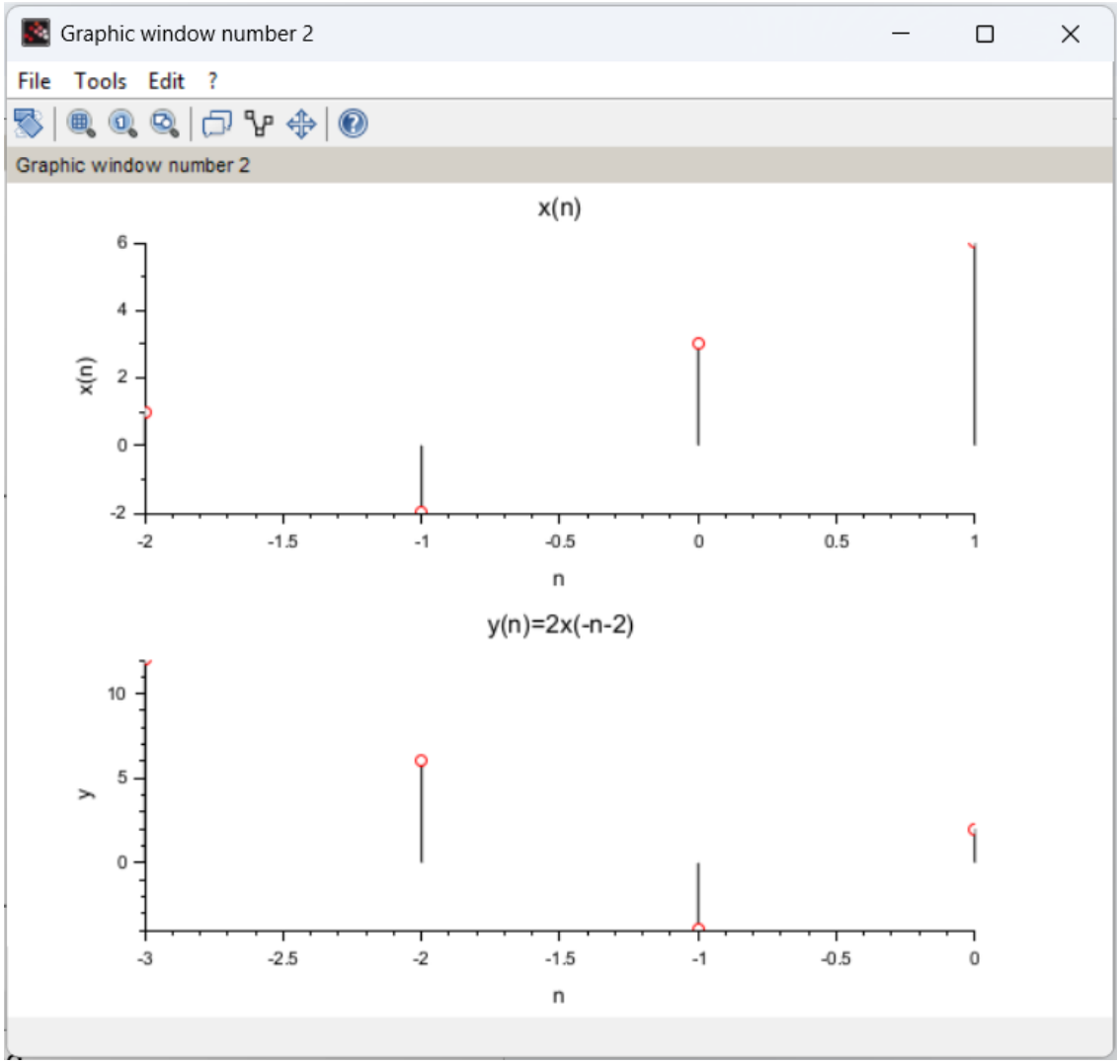
4. Đồ thị



Hình 7: *Đồ thị tín hiệu $x(n)$ và tín hiệu sau phép lật thời gian $y_1(n) = x(-n)$*



Hình 8: *Đồ thị tín hiệu $x(n)$ và tín hiệu sau phép dịch thời gian $y_2(n) = x(n+3)$*



Hình 9: *Đồ thị tín hiệu $x(n)$ và tín hiệu sau phép biến đổi $y_3(n) = 2x(-n-2)$*

5. Nhận xét

- Phép lật thời gian $x(-n)$ đảo ngược tín hiệu quanh gốc tọa độ $n = 0$.
- Phép dịch thời gian làm thay đổi vị trí của tín hiệu trên trục thời gian nhưng không làm thay đổi biên độ.
- Phép nhân biên độ (scaling) làm thay đổi độ lớn của tín hiệu nhưng không ảnh hưởng đến vị trí thời gian.
- Các phép biến đổi này là những phép toán cơ bản trong xử lý tín hiệu số và thường được sử dụng để phân tích đặc tính của hệ thống.

Tài liệu

[1] Scilab Enterprises. (2010). *Scilab for beginners*. Orsay, France: Scilab Enterprises. Available at: https://www.scilab.org/sites/default/files/Scilab_beginners.pdf

[2] Scilab Enterprises. (2024). *Scilab — Open source software for numerical computation*. Available at: https://www.scilab.org

[3] Proakis, J. G., & Manolakis, D. G. (2007). *Digital signal processing: Principles, algorithms, and applications* (4th ed.). Upper Saddle River, NJ: Pearson Prentice Hall.

[4] Link Github. (2025). *Signal Processing Exercises using Scilab*. GitHub repository. Available at: https://github.com/anhthu057/XU_LY_TIN_HIEU_SO/tree/lab2